# 6.17 CloudTrail

Topic 17 of 18 | ~4 hours | Difficulty: Intermediate

## 1 What it is

**AWS CloudTrail** records **who did what, when, from where** in your AWS account. Every call to an AWS API — whether it comes from the console, the AWS CLI, an SDK, Terraform, or another AWS service acting for you — is an **event**: a JSON document naming the caller (`userIdentity`), the action (`eventName`, such as `RunInstances`), the service (`eventSource`), the parameters, the source IP, the user agent, the time and, if it failed, an `errorCode`.

CloudTrail is on by default in a minimal form: **Event history** shows the last **90 days** of *management events* in each region, free, and you can query it from the console or with `aws cloudtrail lookup-events`. For anything longer-lived, or for object-level S3 activity, you create a **trail** that delivers events as compressed JSON files to an S3 §6.8 bucket (and optionally to CloudWatch Logs §6.18), or you use **CloudTrail Lake**, a managed event store you can query with SQL.

## 2 Why it matters in DevOps

- **Incident response and forensics:** "Who opened port 22 to the world?", "Who deleted the bucket?", "Which access key created these 40 GPU instances?" can only be answered from CloudTrail. Without it, an account compromise has no timeline.
- **Debugging IAM:** an `AccessDenied` event names the exact action and principal that failed — often the fastest way to fix a missing permission in IAM §6.7 (and the base for tools that generate least-privilege policies from observed activity).
- **Compliance:** PCI-DSS, SOC 2, HIPAA and ISO 27001 audits require a tamper-evident record of administrative activity and access to sensitive data.
- **Automation:** CloudTrail events feed EventBridge rules, GuardDuty and AWS Config, which can alert or auto-remediate (for example, close a security group that was opened to `0.0.0.0/0`).
- **Cost control:** it also answers "who created this expensive resource?" when the bill jumps.

## 3 Real-world system example

**AWS's own security services.** Amazon GuardDuty analyzes CloudTrail management events (among other sources) to flag suspicious API activity such as credentials used from unusual locations, and IAM Access Analyzer can generate a least-privilege policy by looking at the CloudTrail history of a role — both are documented AWS features. AWS also recommends, in its security best practices and in the AWS Control Tower landing-zone design, an **organization trail** in a dedicated log-archive account whose bucket workload accounts cannot modify. Teams that follow the AWS Well-Architected Security pillar typically combine three things: a multi-region trail, log-file integrity validation, and alerts on high-risk events (root-account use, console login without MFA, changes to trails and security groups). For the SRE and audit context, see DevSecOps §0.12 and Observability §0.10.

## 4 Diagram

From API call to investigation:

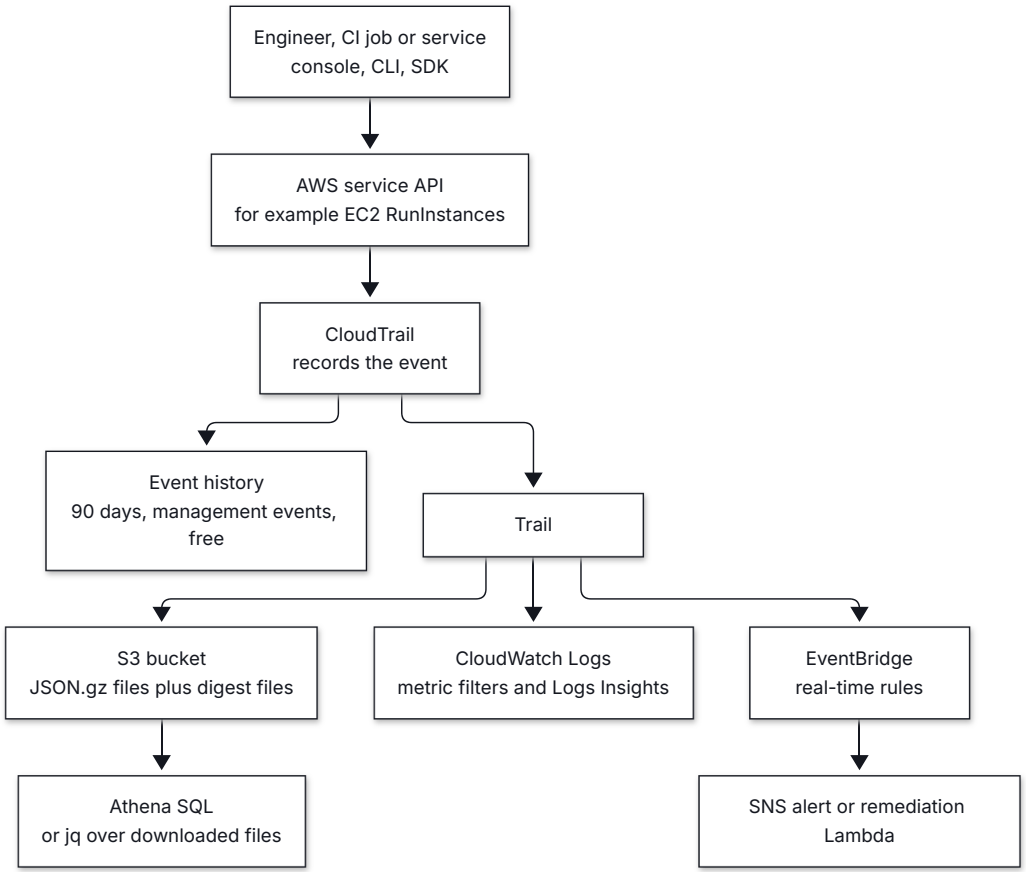



Figure 6.17-1

Anatomy of what you will read most often:

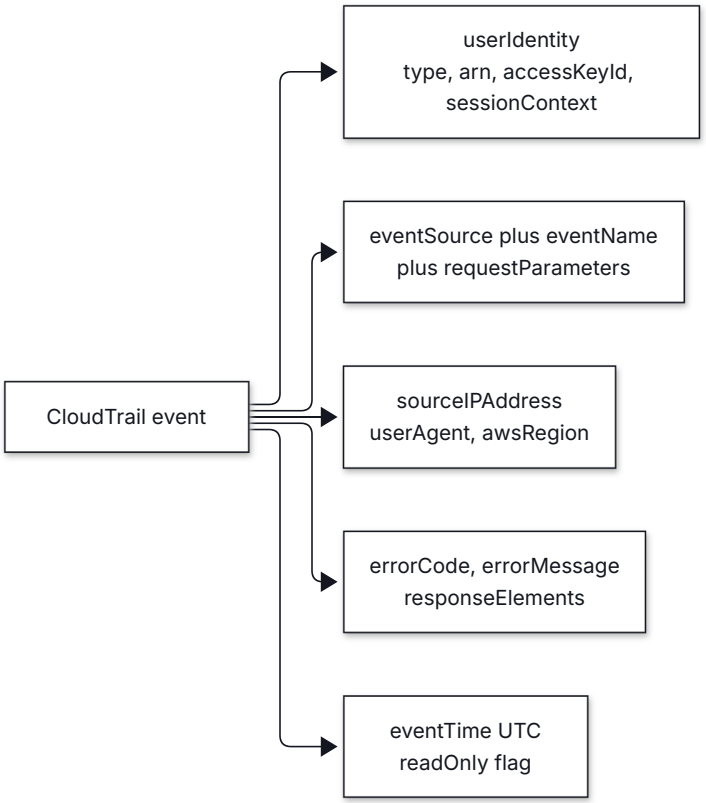



Figure 6.17-2

## 5 How it works

### Event types

| Type | What it records | Examples | Default |
|---|---|---|---|
| **Management events** (control plane) | Operations that create, change or delete resources, plus sign-in and security-config activity | `RunInstances`, `CreateBucket`, `PutBucketPolicy`, `AttachRolePolicy`, `ConsoleLogin`, `AssumeRole` | On: last 90 days in Event history. A trail's first copy of management events per region is free |
| **Data events** (data plane) | High-volume operations on the resources' contents | S3 `GetObject` / `PutObject` / `DeleteObject`, Lambda `Invoke`, DynamoDB item operations | **Off**; billed per event once enabled |

| Type | What it records | Examples | Default |
|---|---|---|---|
| **Insights events** | Statistically unusual API call volume or error rates | a spike of `RunInstances` | Off; extra charge |
| **Network activity events** | VPC endpoint activity (newer feature) | calls through an endpoint | Off; extra charge |

Management events split into **read** (`DescribeInstances`, `ListBuckets`) and **write** (`TerminateInstances`) events; a trail can record both or just one.

### Where events go

- **Event history:** per-region, 90 days, management events only, free; the console and `lookup-events` (a few requests per second at most). Good for a quick question, not for an audit record.
- **Trail to S3:** continuous delivery of gzipped JSON to `s3://bucket/AWSLogs/<account-id>/CloudTrail/<region>/<yyyy>/<mm>/<dd>/`. Delivery typically lags the API call by around five minutes (up to about fifteen). A **multi-region trail** collects all regions, including the global services (IAM, STS, CloudFront) that report in `us-east-1`; an **organization trail** (from the management account) covers every account in AWS Organizations.
- **Trail to CloudWatch Logs:** lets you use metric filters, alarms and Logs Insights near real time (CloudWatch §6.18).
- **CloudTrail Lake:** an event data store that you query with SQL in the console or API; retention you choose; billed for ingestion and queries. A good alternative to building Athena tables.

**Integrity.** Turn on **log file validation**: CloudTrail writes hourly **digest files** that are signed (SHA-256 hashes, RSA signature), and `aws cloudtrail validate-logs` proves that log files were not modified or deleted. Encrypt the logs with KMS §6.15 (SSE-KMS) and protect the bucket: block public access, deny deletes except to a break-glass role, optionally enable S3 Object Lock or MFA delete, and keep the bucket in a separate security account.

**The anatomy of an event** (abridged):

```json
{
  "eventTime": "2026-03-01T02:20:00Z",
  "eventSource": "ec2.amazonaws.com",
  "eventName": "AuthorizeSecurityGroupIngress",
  "awsRegion": "us-east-1",
  "sourceIPAddress": "203.0.113.9",
  "userAgent": "aws-cli/2.x ...",
  "userIdentity": { "type": "IAMUser", "arn": "arn:aws:iam::111122223333:user/alice",
  "accessKeyId": "AKIAIOSFODNN7EXAMPLE" },
  "requestParameters": { "groupId": "sg-0123456789abcdef0", "ipPermissions": { "items": [
  { "fromPort": 22, "ipRanges": { "items": [ { "cidrIp": "0.0.0.0/0" } ] } } ] } },
  "readOnly": false
}
```

Things to know when reading: `userIdentity.type` is `Root`, `IAMUser`, `AssumedRole`, `AWSService` or `FederatedUser`; for `AssumedRole` the `sessionContext` shows who assumed it; `errorCode` of `AccessDenied`, `UnauthorizedOperation` or `Client.…` means the call failed; times are **UTC**; some secrets in request parameters are redacted (`HIDDEN_DUE_TO_SECURITY_REASONS`).

**Cost.** Event history is free. For trails, the first copy of management events in a region is free; additional trails and data events are billed (data events on the order of a few cents to $0.10 per 100,000 events — check the pricing page), plus S3 storage and any Athena/CloudWatch Logs/Lake charges. The classic bill shock is enabling S3 data events on a busy bucket — scope them to the buckets that matter with advanced event selectors.

## 6 Step-by-step guide

**Prerequisites:** AWS CLI v2, `jq`. **Cost warning:** one multi-region trail with only management events costs nothing for CloudTrail itself; the S3 storage for lab logs is cents. Enabling data events on a bucket is billed per event — keep that part short and turn it off. Keep your billing alarm in place (CloudWatch §6.18).

> **No AWS account / LocalStack:** CloudTrail is not meaningfully available in the free LocalStack tier. Steps 1 and 2 let you practise the *investigation* skills on a sample file with `jq`, and everything else needs a real account (it costs nothing in the lab configuration).

1. **Practise on a sample log first (free, offline).**

```bash
cat > sample-trail.json <<'EOF'
{"Records":[
{"eventTime":"2026-03-
  01T02:14:07Z","eventSource":"signin.amazonaws.com","eventName":"ConsoleLogin","sourc
  eIPAddress":"198.51.100.23","userIdentity":
  {"type":"IAMUser","userName":"alice","arn":"arn:aws:iam::111122223333:user/alice"},"
  responseElements":{"ConsoleLogin":"Failure"}},
{"eventTime":"2026-03-
  01T02:15:11Z","eventSource":"signin.amazonaws.com","eventName":"ConsoleLogin","sourc
  eIPAddress":"203.0.113.9","userIdentity":
  {"type":"IAMUser","userName":"alice","arn":"arn:aws:iam::111122223333:user/alice"},"
  responseElements":{"ConsoleLogin":"Success"}},
{"eventTime":"2026-03-
  01T02:20:00Z","eventSource":"ec2.amazonaws.com","eventName":"AuthorizeSecurityGroupI
  ngress","sourceIPAddress":"203.0.113.9","userIdentity":
  {"type":"IAMUser","userName":"alice","arn":"arn:aws:iam::111122223333:user/alice"},"
  requestParameters":{"groupId":"sg-0123456789abcdef0","ipPermissions":{"items":
  [{"ipProtocol":"tcp","fromPort":22,"toPort":22,"ipRanges":{"items":
  [{"cidrIp":"0.0.0.0/0"}]}}]}}},
{"eventTime":"2026-03-
  01T02:21:00Z","eventSource":"s3.amazonaws.com","eventName":"DeleteBucket","errorCode
  ":"AccessDenied","sourceIPAddress":"203.0.113.9","userIdentity":
  {"type":"IAMUser","userName":"alice","arn":"arn:aws:iam::111122223333:user/alice"}}
]}
EOF
# Failed console logins
jq -r '.Records[] | select(.eventName=="ConsoleLogin" and
  .responseElements.ConsoleLogin=="Failure") |
  [.eventTime,.userIdentity.userName,.sourceIPAddress] | @tsv' sample-trail.json
# Security groups opened to the world
jq -r '.Records[] | select(.eventName=="AuthorizeSecurityGroupIngress") |
  select(.requestParameters.ipPermissions.items[].ipRanges.items[].cidrIp=="0.0.0.0/0"
  ) | [.eventTime,.userIdentity.arn,.requestParameters.groupId] | @tsv' sample-
  trail.json
# Everything that was denied
jq -r '.Records[] | select(.errorCode != null) | [.eventTime,.eventName,.errorCode] |
  @tsv' sample-trail.json
```

Expected: one failed login from `198.51.100.23`, one world-open SSH rule from `alice` on `sg-0123456789abcdef0`, and one `DeleteBucket AccessDenied`. Together these tell a story: a failed login, then success from a different IP, then an opened firewall and an attempted deletion — the shape of an account takeover.

2. **Query real Event history.**

```bash
export AWS_REGION=us-east-1
aws cloudtrail lookup-events --lookup-attributes
  AttributeKey=EventName,AttributeValue=ConsoleLogin \
  --max-results 5 --query 'Events[].[EventTime,Username,EventName]' --output table
aws cloudtrail lookup-events --lookup-attributes
  AttributeKey=EventSource,AttributeValue=ec2.amazonaws.com \
  --start-time "$(date -u -d '-1 day' +%FT%TZ)" --max-results 20 \
  --query 'Events[].[EventTime,EventName,Username]' --output table
```

(`date -d` is GNU; on macOS use `date -u -v-1d +%FT%TZ`.) `lookup-events` supports one attribute at a time (`EventName`, `Username`, `EventSource`, `ResourceName`, `ResourceType`, `AccessKeyId`, `ReadOnly`, `EventId`). Pull one full event: `... --max-results 1 --query 'Events[0].CloudTrailEvent' --output text | jq .`

3. **Create a log bucket and a multi-region trail with validation.**

```bash
```

```
ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
LOGBUCKET=shop-trail-logs-$ACCOUNT_ID
TRAIL=shop-lab-trail
aws s3api create-bucket --bucket $LOGBUCKET
aws s3api put-public-access-block --bucket $LOGBUCKET --public-access-block-
  configuration \

  BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuck
  ets=true
cat > trail-policy.json <<EOF
{ "Version": "2012-10-17", "Statement": [
  { "Sid": "AWSCloudTrailAclCheck", "Effect": "Allow", "Principal": { "Service":
  "cloudtrail.amazonaws.com" },
    "Action": "s3:GetBucketAcl", "Resource": "arn:aws:s3:::$LOGBUCKET",
    "Condition": { "StringEquals": { "aws:SourceArn": "arn:aws:cloudtrail:us-east-
  1:$ACCOUNT_ID:trail/$TRAIL" } } },
  { "Sid": "AWSCloudTrailWrite", "Effect": "Allow", "Principal": { "Service":
  "cloudtrail.amazonaws.com" },
    "Action": "s3:PutObject", "Resource":
  "arn:aws:s3:::$LOGBUCKET/AWSLogs/$ACCOUNT_ID/*",
    "Condition": { "StringEquals": { "s3:x-amz-acl": "bucket-owner-full-control",
                                     "aws:SourceArn": "arn:aws:cloudtrail:us-east-
  1:$ACCOUNT_ID:trail/$TRAIL" } } } ] }
EOF
aws s3api put-bucket-policy --bucket $LOGBUCKET --policy file://trail-policy.json
aws cloudtrail create-trail --name $TRAIL --s3-bucket-name $LOGBUCKET \
  --is-multi-region-trail --include-global-service-events --enable-log-file-validation
aws cloudtrail start-logging --name $TRAIL
aws cloudtrail get-trail-status --name $TRAIL --query
  '{Logging:IsLogging,LastDelivery:LatestDeliveryTime}'
```

Expected: `IsLogging: true`. Delivery to S3 starts within about 5-15 minutes.

4. **Generate some activity, then read the files.**

```
BASH
aws ec2 create-security-group --group-name trail-demo --description "trail demo" >
  /dev/null     # a write event
aws ec2 delete-security-group --group-name trail-demo
sleep 600     # or come back later
aws s3 sync s3://$LOGBUCKET/AWSLogs/$ACCOUNT_ID/CloudTrail/ ./trail-logs/ --quiet
find ./trail-logs -name '*.json.gz' | head -3
find ./trail-logs -name '*.json.gz' -exec gunzip -c {} + \
  | jq -r '.Records[] | select(.eventName | test("SecurityGroup")) |
  [.eventTime,.eventName,.userIdentity.arn,.sourceIPAddress] | @tsv'
```

5. **Validate integrity.**

```bash
aws cloudtrail validate-logs --trail-arn arn:aws:cloudtrail:us-east-
  1:$ACCOUNT_ID:trail/$TRAIL \
  --start-time "$(date -u -d '-2 hours' +%FT%TZ)"
```

Expected: lines saying digest files and log files were valid (log delivery must have occurred and a digest must have been written, which can take an hour).

6. **Investigation queries on a larger scale.**
   - *Athena:* in the console, *CloudTrail → Event history → Create Athena table* (the console fills in the standard table definition for your bucket). Then, for example:

```sql
SELECT eventtime, useridentity.arn, eventname, sourceipaddress, errorcode
FROM cloudtrail_logs
WHERE eventsource = 'ec2.amazonaws.com' AND eventname =
  'AuthorizeSecurityGroupIngress'
  AND eventtime > '2026-03-01T00:00:00Z'
ORDER BY eventtime DESC LIMIT 50;
```

   - *CloudWatch Logs Insights* (if the trail also writes to a log group):

```sql
fields @timestamp, eventName, userIdentity.arn, sourceIPAddress, errorCode
| filter errorCode like /AccessDenied|UnauthorizedOperation/
| stats count(*) as denied by userIdentity.arn, eventName
| sort denied desc | limit 20
```

Useful standing questions: *root account usage* (`userIdentity.type = 'Root'`), *console login without MFA* (`additionalEventData` shows `MFAUsed: "No"`), *who changed IAM* (`eventsource = 'iam.amazonaws.com'` and `readonly = 'false'`), *who touched the trail* (`StopLogging`, `DeleteTrail`, `UpdateTrail`).

7. **Optional: S3 data events for one bucket** (billed per event; do this briefly).

```bash
cat > selectors.json <<EOF
[{ "Name": "S3 objects in the lab log bucket only",
   "FieldSelectors": [ { "Field": "eventCategory", "Equals": ["Data"] },
                       { "Field": "resources.type", "Equals": ["AWS::S3::Object"] },
                       { "Field": "resources.ARN", "StartsWith":
  ["arn:aws:s3:::$LOGBUCKET/"] } ] }]
EOF
aws cloudtrail put-event-selectors --trail-name $TRAIL --advanced-event-selectors
  file://selectors.json
```

   To turn data events off again, switch back to a basic management-only selector: `aws cloudtrail put-event-selectors --trail-name $TRAIL --event-selectors '[{"ReadWriteType":"All","IncludeManagementEvents":true}]'` (or simply delete the trail in step 8).

8. **Cleanup (do not skip).**

```bash
aws cloudtrail stop-logging --name $TRAIL
aws cloudtrail delete-trail --name $TRAIL
aws s3 rm s3://$LOGBUCKET --recursive
aws s3api delete-bucket --bucket $LOGBUCKET
rm -rf trail-logs sample-trail.json trail-policy.json selectors.json
aws cloudtrail describe-trails --query 'trailList[].Name'     # shop-lab-trail must be
  gone
```

   In a real account you keep at least one multi-region management-event trail running permanently; deleting it here is for the lab only.

## 7 Real-world scenario

*Illustrative scenario.* On a Saturday the `shop-api` account's bill alarm (CloudWatch §6.18) fires: charges are 10x the normal daily amount. **Investigation:** in Event history, filter `EventName = RunInstances` for the last 24 hours — 60 `p3.16xlarge` instances in `ap-southeast-2`, a region the team never uses, launched using the access key of a CI user, from an IP address that is not the CI runner's. `userAgent` is a Python SDK, not the Terraform user agent the pipeline normally has. The key was leaked in a public repository. **Fix:** deactivate the access key, terminate the instances, check for `CreateUser`, `CreateAccessKey`, `AttachUserPolicy` events from the same key (none, luckily), rotate every secret that CI could read (Secrets Manager §6.16), and file an AWS support request for the charges. **Follow-ups:** a service control policy denying unused regions, EventBridge alerting on `RunInstances` in unexpected re-

gions, and moving CI to short-lived credentials via OIDC. **Lesson:** you can only investigate what was recorded; a multi-region trail with a protected log bucket is the first control every account gets.

## 8 Common mistakes and best practices

**Don't**

- Rely on Event history alone: 90 days, management events only, per region, not tamper-evident.
- Create a single-region trail and assume you are covered; attackers use regions you never look at.
- Store the logs in the same account and bucket the attacker can reach with the stolen credentials, with delete permission.
- Enable data events on every bucket and Lambda "just in case" — scope them or the bill and the noise will explode.
- Forget that delivery is delayed by minutes; do not use CloudTrail S3 files for real-time alerting (use EventBridge or CloudWatch Logs).
- Put secrets in request parameters (for example tags or environment variables) and then act surprised when they appear in logs.

**Do**

- Run an **organization or account multi-region trail** with log file validation, SSE-KMS and a locked-down bucket from day one.
- Send the trail to CloudWatch Logs and create alarms for root usage, console logins without MFA, IAM policy changes, security group and route-table changes, and `StopLogging` / `DeleteTrail`.
- Use consistent user agents / role session names for automation ( `RoleSessionName=github-actions-<run-id>` ) so events are attributable.
- Keep logs per your retention policy (S3 lifecycle to cheaper storage) and practise Athena or Lake queries before an incident.
- Define the trail as code (Terraform [§7.1]) and review changes to it.

## 9 Tasks

- [ ] **T1 — Read an event** *(≈30 min, beginner)*
  - **Goal:** get comfortable with the event structure.
  - **Steps:**

1. Do something visible in the console (create a security group), wait a few minutes.
2. Find the event with `lookup-events --lookup-attributes AttributeKey=EventName,AttributeValue=CreateSecurityGroup`.
3. Print the full `CloudTrailEvent` JSON and identify who, what, from where, and the request parameters.

- **Done when:** you can point at `userIdentity.arn`, `eventName`, `sourceIPAddress`, `userAgent` and the new group's ID in the response.

- [ ] **T2 — Investigate the sample incident** *(≈30 min, beginner)*
  - **Goal:** reconstruct a timeline with `jq`.
  - **Steps:**
    1. Run the three `jq` queries from step 1 on the sample file.
    2. Add a fourth query listing every event by `sourceIPAddress`, sorted by time.
    3. Write a five-line incident timeline.
  - **Done when:** your timeline lists the failed login, successful login from another IP, the world-open SSH rule and the denied `DeleteBucket`, each with a timestamp.

- [ ] **T3 — Create a protected trail** *(≈45 min, intermediate)*
  - **Goal:** durable, validated, multi-region logging.
  - **Steps:**
    1. Follow step 3 to create the bucket, policy and trail with log file validation.
    2. Generate activity (create and delete a security group) and wait for delivery.
    3. Run `get-trail-status`, list the S3 prefix, and `validate-logs`.
  - **Done when:** `.json.gz` files exist in your bucket and `validate-logs` reports valid digests and files.

- [ ] **T4 — Query it with Athena or Logs Insights** *(≈60 min, intermediate)*
  - **Goal:** answer audit questions in SQL.
  - **Steps:**
    1. Create the Athena table from the console (or send the trail to a CloudWatch Logs group).
    2. Write queries for: all `AccessDenied` events by principal; all IAM write events; any `ConsoleLogin` without MFA.
    3. Save the queries in a `queries.md` file in your repo.
  - **Done when:** each query returns rows (generate a denied call by attempting a forbidden action to be sure) and you can explain each result.

- [ ] **T5 — Alert on dangerous events** *(≈60 min, advanced)*
  - **Goal:** turn the log into a detector.
  - **Steps:**
    1. Create an EventBridge rule on `source: aws.ec2`, `detail-type: AWS API Call via CloudTrail`, `eventName: AuthorizeSecurityGroupIngress` with an SNS email target.
    2. Open a security group port to `0.0.0.0/0` in a test group.
    3. Remove the rule and delete the test group, the rule and the topic.
  - **Done when:** you received an email within a couple of minutes naming the principal and group, and all lab resources are deleted.
- [ ] **T6 — Prove it: tamper test and teardown** *(≈45 min, advanced)*
  - **Goal:** show that integrity validation detects tampering.
  - **Steps:**
    1. Without the guide, pick one delivered log file, download it, change a field and re-upload it to the same key (on your lab bucket only).
    2. Run `validate-logs` for that window and read the output.
    3. Explain why this protects an audit, then run the cleanup (step 8).
  - **Done when:** `validate-logs` reports the modified file as invalid (hash mismatch) and `describe-trails` shows your lab trail is gone.

## 10 Self-check questions

1. What is the difference between management events and data events, and which are recorded by default?
2. Why isn't Event history enough for a production account?
3. How do you prove that CloudTrail log files were not altered?
4. A user says "something deleted my S3 object last week" — which kind of event do you need, and was it enabled by default?
5. How would you find which access key launched unexpected EC2 instances?
6. Why should the trail bucket live in a different account from the workloads?

**ANSWERS**

1. Management events are control-plane operations (create, modify, delete resources, sign-ins) and are recorded by default in Event history; data events are high-volume object-level operations (S3 objects, Lambda invokes, DynamoDB items) and must be enabled on a trail.

2. It keeps 90 days, only management events, per region, can't be exported as a tamper-evident record, and is limited in querying.
3. Enable log file validation and run `aws cloudtrail validate-logs`, which checks the signed digest files against the log files.
4. An S3 data event (`DeleteObject`); it is off by default, so unless it was enabled for that bucket you may have no record (S3 server access logs or versioning might still help).
5. `lookup-events` with `EventName=RunInstances` and read `userIdentity.accessKeyId`, `sourceIPAddress` and `userAgent` (or query Athena/Lake for the same).
6. A compromised workload account's credentials then cannot delete or alter the evidence.

## 11 Further reading

- *AWS CloudTrail User Guide* (docs.aws.amazon.com/awscloudtrail) — event reference, trails, Lake, log file integrity validation.
- *AWS CloudTrail pricing* page — current charges for data events, Insights and Lake.
- *AWS Security Incident Response Guide* — using CloudTrail in investigations.
- *AWS Well-Architected Framework: Security pillar* — detection and logging practices.